

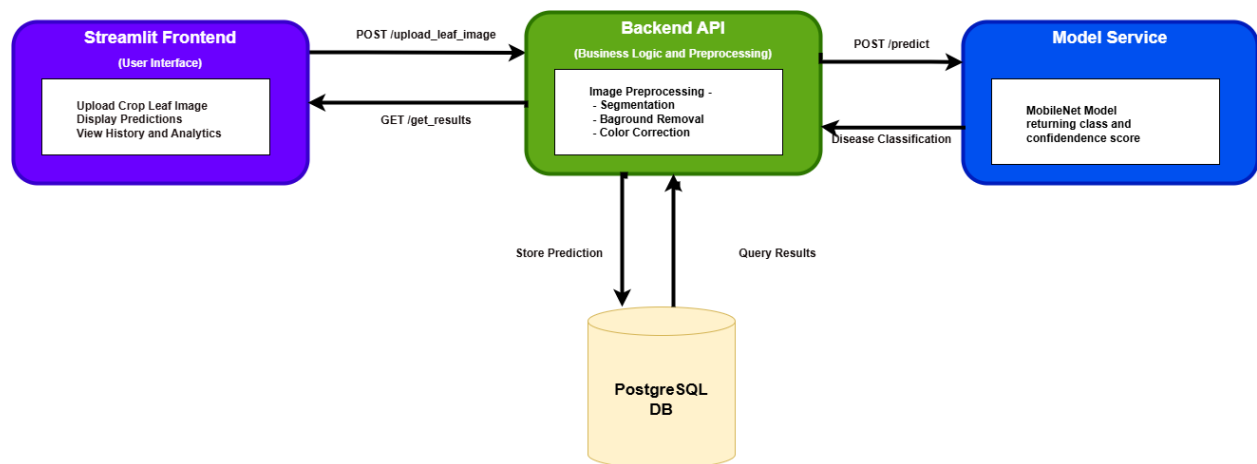
Crop Leaf Disease Detection System

1. Overview

This project provides an end-to-end system for classifying crop leaf health using computer vision and deep learning, exposed via REST APIs. It covers:

- Image preprocessing (segmentation, background removal, color correction)
- AI inference (MobileNet model)
- Results storage (PostgreSQL)
- REST endpoints for prediction and history

2. System Architecture



The system consists of three main components:

- **Frontend:** Streamlit-based user interface for image upload and results visualization.
- **Backend API:** FastAPI service handling business logic, preprocessing, and database operations.
- **Model Service:** Dedicated FastAPI service for deep learning inference.

3. Project Structure

```
/
├── model-service/
│   ├── app/main.py      # FastAPI app: model inference endpoint
│   └── models/          # .keras trained model file
```

```
├── requirements.txt
├── backend/
│   ├── app/
│   │   ├── main.py          # FastAPI app: all business logic endpoints
│   │   ├── api/endpoints.py  # /upload_leaf_image and /get_results
│   │   ├── preprocessing/    # image processing module
│   │   └── db/database.py     # SQLAlchemy models, DB setup
│   └── requirements.txt
├── frontend/
│   ├── streamlit_app.py      # Streamlit interface
│   └── requirements.txt
├── ml_training/              # (not deployed) model training scripts
├── infra/
└── README.md
```

4. Database Schema

Predictions Table

The core table for storing prediction results:

Column Name	Data Type	Primary Key	Not NULL	Description
leaf_id	integer	Yes	Yes	Unique prediction ID
crop_type	varchar	No	Yes	Crop type (Tomato, Potato, etc.)
disease_status	varchar	No	Yes	Health/Stage: Healthy, Early_Disease, Severe_Disease
confidence_score	double	No	Yes	Model confidence (0.0 – 1.0)
timestamp	timestamp	No	Yes	Time when prediction stored
image_filename	varchar	No	No	Filename of original uploaded image

5. Core Architecture

5.1. Image Preprocessing

Implemented in backend/app/preprocessing/image_processor.py using OpenCV and Pillow:

- **Color correction:** CLAHE histogram equalization (illumination normalization).
- **Background removal:** HSV thresholding, Gaussian blur, morphological ops, contour extraction.

- **Segmentation:** Largest contour selected, image cropped and resized (224x224).
- **Exposed via:** Called as a utility in /upload_leaf_image endpoint.

5.2. Model Service

- FastAPI service (model-service/app/main.py)
- Loads pre-trained Keras .keras model (MobileNet, outputs 3-class)
- Accepts preprocessed arrays, returns predicted class and confidence
- Maps fine-grained class (from model) into: Healthy, Early Disease, Severe Disease

5.3. Backend API

Implemented in backend/app/main.py and backend/app/api/endpoints.py:

1. /upload_leaf_image [POST]

- **Accepts:** multipart/form-data (image file)
- **Process:**
 - 1. Applies preprocessing steps
 - 2. Sends processed image to Model Service; receives prediction and confidence
 - 3. Infers crop type (from filename)
 - 4. Stores result with all metadata in Postgres DB (see schema above)
 - 5. Returns JSON response

- **Example Response:**

```
{
  "leaf_id": 1,
  "crop_type": "Tomato",
  "disease_status": "Early_Disease",
  "confidence_score": 0.91,
  "timestamp": "2025-10-20T12:05:01",
  "image_filename": "tomato_leaf_1.jpg"
}
```

2. /get_results [GET]

- **Query params:** limit, crop_type, disease_status, start_date, end_date
- **Features:** - Lists recent/past predictions (paginated) - Summary stats: total predictions, class distribution

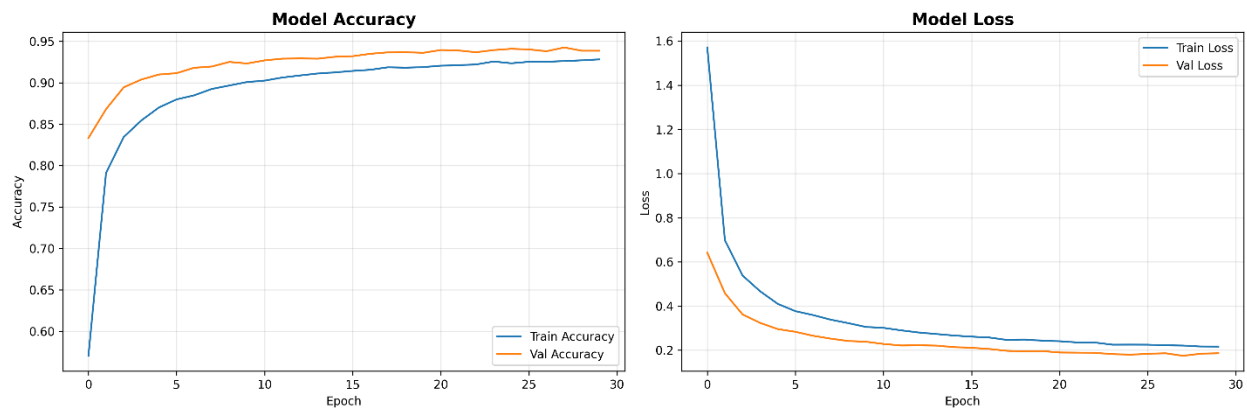
5.4. Frontend

- Implemented in Streamlit (frontend/streamlit_app.py)
- Calls backend /upload_leaf_image for new predictions
- Calls /get_results for fetching and displaying DB history/stats

6. Results

6.1 Model Accuracy and Loss Curves

Accuracy Achieved: ~95%



6.2. Streamlit Frontend

1. Upload and Prediction Interface - Healthy Prediction

 **Crop Leaf Disease Detection**

1 Upload a Leaf Image for Disease Detection

Choose a leaf image (JPEG/PNG, max 10MB)

Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Uploading and analyzing image....

Prediction complete!



Uploaded Leaf

Crop Type: Apple
Disease Status: Healthy
Confidence: 92.12%
Timestamp: 2025-10-20T12:35:53.327863

2. Upload and Prediction Interface - Early Disease Prediction

Crop Leaf Disease Detection

1 Upload a Leaf Image for Disease Detection

Choose a leaf image (JPEG/PNG, max 10MB)

 Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

[Browse files](#)

[Analyze Image](#)

Uploading and analyzing image...

Prediction complete!



Uploaded Leaf

Crop Type: Potato

Disease Status: Early_Disease

Confidence: 99.89%

Timestamp: 2025-10-20T12:11:05.230215

3. Upload and Prediction Interface - Severe Disease Prediction

Crop Leaf Disease Detection

1 Upload a Leaf Image for Disease Detection

Choose a leaf image (JPEG/PNG, max 10MB)

 Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Uploading and analyzing image...

Prediction complete!



Uploaded Leaf

Crop Type: Tomato

Disease Status: **Severe_Disease**

Confidence: 100.00%

Timestamp: 2025-10-20T12:11:38.629736

Users can upload crop leaf images and receive instant disease classification results

4. Prediction Results Display

2 Prediction History & Statistics

Show Prediction History

Results Limit

10

-

+

Filter by Crop Type (optional)

Filter by Disease Status (optional)

Refresh History

Total Predictions: 10

Disease Distribution:

{

"Early_Disease" : 1

"Severe_Disease" : 1

"Healthy" : 8

}

ID: 10 | Crop: Apple | Disease: Healthy | Confidence: 92.12% | Time: 2025-10-20T12:13:26.226777

ID: 9 | Crop: Tomato | Disease: Severe_Disease | Confidence: 100.00% | Time: 2025-10-20T12:11:38.629736

ID: 8 | Crop: Potato | Disease: Early_Disease | Confidence: 99.89% | Time: 2025-10-20T12:11:05.230215

ID: 7 | Crop: Apple | Disease: Healthy | Confidence: 20.73% | Time: 2025-10-20T12:09:45.121446

ID: 6 | Crop: Apple | Disease: Healthy | Confidence: 92.12% | Time: 2025-10-20T12:09:20.835171

ID: 5 | Crop: Apple | Disease: Healthy | Confidence: 82.54% | Time: 2025-10-20T12:08:41.925662

ID: 4 | Crop: Apple | Disease: Healthy | Confidence: 92.12% | Time: 2025-10-20T12:08:07.552023

ID: 3 | Crop: Apple | Disease: Healthy | Confidence: 92.12% | Time: 2025-10-20T11:30:14.630912

ID: 2 | Crop: Apple | Disease: Healthy | Confidence: 57.02% | Time: 2025-10-20T11:29:44.130467

ID: 1 | Crop: Apple | Disease: Healthy | Confidence: 92.12% | Time: 2025-10-20T11:29:19.631402

Detailed prediction showing crop type, disease status, and confidence score

6.3. FastAPI Backend Results

1. Upload Leaf Image Endpoint

Crop Disease Backend API 1.0.0 CAS 3.1

/openapi.json

Predictions

POST /upload_leaf_image Upload Leaf Image

Parameters

No parameters

Request body required

multipart/form-data

file required

string(binary)

Choose File AppleCedarRust1.JPG

Execute

Clear

Responses

request url

https://crop-leaf-disease-detection-1.onrender.com/upload_leaf_image

Server response

Code

Details

200

Response body

```
{
  "leaf_id": 10,
  "crop_type": "Apple",
  "disease_status": "Healthy",
  "confidence_score": 0.9211976538167725,
  "timestamp": "2025-10-20T12:13:26.226777",
  "image_filename": "AppleCedarRust1.JPG"
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
alt-svc: h3="443"; ma=86400
cf-cache-status: DYNAMIC
cf-ray: 9918dcf8be8937d-MAA
content-encoding: br
content-length: 142
content-type: application/json
date: Mon, 20 Oct 2025 12:13:26 GMT
priority: u=1,i
rndr-id: e7842bea-c817-4326
server: cloudflare
```

422

Validation Error

No links

Media type

application/json

Example Value

Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

Interactive API documentation powered by Swagger UI POST /upload_leaf_image endpoint with request/response schemas

2. Get Results Endpoint

GET

/get_results

Get Results

Cancel

Name	Description
limit	10
integer (query)	maximum: 100 minimum: 1
crop_type	crop_type
string (string null) (query)	
disease_status	disease_status
string (string null) (query)	
start_date	start_date
string (string null) (\$date-time) (query)	
end_date	end_date
string (string null) (\$date-time) (query)	

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
  'https://crop-leaf-disease-detection-1.onrender.com/get_results?limit=10' \
  -H 'accept: application/json'
```

Request URL

https://crop-leaf-disease-detection-1.onrender.com/get_results?limit=10

Server response

Code	Details
200	Response body <pre>{ "disease_status": "Healthy", "confidence_score": 0.92137993267725, "timestamp": "2025-10-20T12:13:26.226777", "image_filename": "AppleCedarRust1.JPG" }, { "leaf_id": 3, "crop_type": "Tomato", "disease_status": "Severe_Disease", "confidence_score": 0.899984562792353, "timestamp": "2025-10-20T12:13:26.629736", "image_filename": "TomatoEarlyBlight2.JPG" }, { "leaf_id": 8, "crop_type": "Potato", "disease_status": "Early_Disease", "confidence_score": 0.99931851719002, "timestamp": "2025-10-20T12:13:46.238025", "image_filename": "PotatoEarlyBlight2.JPG" }, { "leaf_id": 7, "crop_type": "Apple", "disease_status": "Healthy", "confidence_score": 0.975083648711689, "timestamp": "2025-10-20T12:09:45.121546", "image_filename": "AppleCedarRust4.JPG" } }</pre> Response headers <pre>alt-svc: h3=":443"; ma=86400 cf-cache-status: DYNAMIC cf-ray: 99186c8065d9937d-MAA content-encoding: br content-length: 409 content-type: application/json date: Mon, 20 Oct 2025 12:13:52 GMT priority: u=1,i render-id: 22ba4772-9ec8-489f server: cloudflare server-timing: cfExtPri vary: Accept-Encoding x-render-origin-server: uvicorn</pre>

Code	Description	Links
200	Successful Response	No links
Media type: application/json Controls Accept header Example Value Schema <pre>"string"</pre>		
422	Validation Error	No links
Media type: application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] }</pre>		

GET /get_results endpoint with query parameters and filtering options

7. Setup Instructions

- **Clone Repository**

```
git clone https://github.com/rasikathakur/Crop_Leaf_Disease_Detection
```

- **Installation**

- Create separate environments for each service (model-service, backend, frontend).
- Install dependencies per requirements.txt in each service's folder.

- **Run Locally**

- **Model Service:**

```
uvicorn app.main:app --host 0.0.0.0 --port 8001
```

- **Backend:**

Set environment variables:

```
export MODEL_SERVICE_URL=http://localhost:8001  
export DATABASE_URL=postgresql://username:password@localhost:5432/db_name
```

Run server:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

- **Frontend:**

Set environment variable:

```
export BACKEND_URL=http://localhost:8000
```

Run Streamlit:

```
streamlit run streamlit_app.py
```

8. Key Points

- **Image preprocessing is critical for accuracy** - proper color correction and background removal significantly improve model performance.
 - **Separated model service** allows for easy scaling, updating, or moving to GPU infrastructure if needed.
 - **Clear schema** ensures robust data storage and easy querying.
 - **Modular project structure** enables easy updates and future extensions.
-

9. Deployed Links

- **Model Service:** <https://crop-leaf-disease-detection.onrender.com/docs>
 - **Backend API:** <https://crop-leaf-disease-detection-1.onrender.com/docs>
 - **Frontend:** <https://crop-leaf-disease-detection-2.onrender.com>
 - **Source Code:** https://github.com/rasikathakur/Crop_Leaf_Disease_Detection
-